

多智能体协作、通信与协议

并行、隔离、委派、共享状态、失败传播与跨Agent互操作

研究范围	2025-08-20 至 2026-08-19; 另列少量边界基线
语料规模	重点分析 6 篇多智能体论文, A2A/MCP/Skills三类协议边界
证据原则	优先论文原文、作者项目页、官方工程博客和GitHub仓库; 主张与推断分开
版本	v1.0 生成日期 2026-08-19

本报告不是排行榜。它是一套研究地图:

解释哪些结果可以相信、哪些只是方向性证据、哪些工程选择会改变结论。

本卷导航

01	多智能体何时真的有用
02	拓扑、角色与信息边界
03	通信协议与共享状态
04	成本、重复与错误传播
05	失败定位与恢复
06	可复用工程模式
07	案例与代表论文精读卡

1. 多智能体何时真的有用

多智能体不是“更智能”的同义词。它是把任务和信息分配到多个独立执行上下文中的系统设计。只有当独立性、并行性、专业能力或权限隔离产生可测价值时，额外Agent才值得成本。

场景	前提	收益	主要代价
可并行搜索	多个来源/假设相互独立	墙钟时间下降; 证据覆盖增加	重复搜索和结果合并
专业工具	不同Agent拥有独占工具/数据	最小权限和领域提示	路由错误与能力隐藏
上下文隔离	子任务材料大且互相干扰	减少上下文污染	摘要交接损失
独立复核	结果可由另一策略验证	降低同一路径盲点	共享模型导致相关错误
组织映射	职责/批准权真实存在	责任和升级路径清晰	把人类组织缺陷复制给Agent
探索多样性	存在多个合理策略	保留替代方案	投票可能压制少数正确意见

不应使用多Agent的场景

- 任务短、动作少、一个Agent能在单一上下文内完成。
- 所有Agent使用相同模型、相同信息、相同工具且只是改角色名。
- 结果没有可合并结构，只能让另一个模型重新读全部内容。
- 共享状态没有版本控制或所有者，多个Agent会互相覆盖。
- 成本、延迟或审计要求严格，无法承受多倍调用和消息。

2. 拓扑、角色与信息边界

拓扑	机制	优点	缺点
主管-工作者	中心分解和合并	控制清晰、易审计	主管成为瓶颈和单点错误
流水线	固定阶段交付	结构稳定、接口明确	上游错误向后传播
黑板	共享任务/证据空间	异步协作和增量整合	冲突、陈旧和权限复杂
对等网络	Agent互相请求/协商	灵活、无中心瓶颈	消息爆炸和责任模糊
市场/竞标	按能力/成本选择Agent	动态路由	报价/能力声明难验证
辩论/评审	独立方案后互相批评	发现盲点	相关偏差、说服力替代正确性
团队分支	并行在隔离分支产出	适合代码/文档合并	接口和集成测试决定上限

角色应由能力和责任定义

一个有效角色至少包含： 可见信息、可用工具、可修改对象、交付schema、验收人、预算、超时和升级条件。仅写“你是资深架构师”不会创造专业化；给它架构文档、依赖图、只读代码扫描工具和必须输出的ADR结构，才形成真正角色。

信息隔离的两面

隔离能降低干扰和权限风险，也会让Agent缺少关键上下文。正确做法不是完全隔离或完全共享，而是最小充分共享： 共享目标、接口、决定、证据引用和状态版本；不共享无关原始轨迹、敏感数据和未验证推断。子Agent返回的应是结构化产物，不是整段聊天。

3. 通信协议与共享状态

机制	连接	载荷	作用	风险
MCP	Agent到工具/资源	工具schema、资源、提示	调用能力	工具供应链、权限、恶意输
A2A	Agent应用到Agent应用	任务、消息、状态、产物、Agent Card	跨框架委派	身份、能力真实性、SLA、
Skills	Agent到可加载能力包	说明、代码、资源、适用条件	程序知识复用	版本、来源、沙箱与依赖
内部事件总线	运行时组件	事件、状态变化、追踪	解耦和重放	schema演化、顺序、一致

建议的消息信封

字段	目的
task_id / parent_id	任务树和因果链
sender / capability	发送者身份与声明能力
goal / constraints	目标和不可违反条件
input_refs	输入对象引用、版本和访问范围
output_schema	必须返回的字段和证据
claims	事实/推断/建议分型，带来源和置信度
effects	已执行副作用、对象ID、版本和可回滚性
status	working/blocked/failed/completed/needs_human
budget	token、时间、工具和成本余额
trace_ref	完整轨迹位置，按需读取而非默认广播

协议不是信任
A2A能让两个系统互相说话，但不能证明对方能完成任务；MCP能让Agent发现工具，但不能证明工具安全；Skill包能被加载，但不能证明适用于当前环境。能力必须通过验证任务、权限租约、来源和运行时监控建立。

4. 成本、重复与错误传播

多Agent的成本不是Agent数量乘模型价格这么简单。还包括任务分解、上下文复制、重复检索、消息压缩、合并、冲突解决、评审和失败返工。应以每个可验证成功的总成本衡量，而不是总token或墙钟时间单项。

成本项	内容	设计含义
推理	各Agent输入/输出和缓存	独立上下文可降低单次长度，但增加总调用
工具	检索、浏览、代码、外部API	并行可缩短时间，也可能触发配额和重复
协调	路由、消息、摘要、投票、合并	通常被论文和产品演示低估
执行	沙箱、容器、浏览器、存储	并行峰值资源高
验证	测试、裁判、人工审阅	高质量系统的必要成本
失败	重试、回滚、清理、事故	尾部成本可能主导总体

错误传播路径

- 委派错误: 主管把任务交给错误能力或遗漏关键约束。
- 执行错误: 工作者选错工具、参数或状态。
- 摘要错误: 错误在交接时被压缩为确定事实。
- 共识错误: 多个相似模型互相强化相同偏差。
- 合并错误: 局部正确产物在接口、版本或假设上不兼容。
- 恢复错误: 系统只重试失败Agent，而未回滚已经污染的共享状态。

如何测量多Agent增益

至少报告单Agent强基线、相同总token基线、相同墙钟预算、相同工具和相同模型。对每个子Agent做消融，统计独有证据、重复工作、合并冲突和净成功贡献。若多Agent只在总token增加时提高，应把它归因于测试时计算，而不是“协作智能”。

5. 失败定位与恢复

问题	诊断	输出
谁	哪个Agent首次产生错误声明或动作？	责任Agent定位
何时	哪一步后系统已无法在剩余预算恢复？	最早决定性步骤
什么	工具、参数、计划、消息、状态或验证错误？	失败类型
为何	缺信息、错误信息、模型限制、权限、环境还是协议？	根因
影响	污染了哪些共享状态、下游产物和外部对象？	影响图
恢复	重试、换Agent、重规划、回滚、补偿或人工？	恢复策略

多视角验证

AgentLocate类方法用多个评估视角聚合定位，优点是减少单一裁判盲点。生产系统还应加入确定性信号：工具日志、状态diff、测试、版本冲突和权限拒绝。LLM判断适合解释开放语义，不能替代可执行证据。

恢复必须处理共享状态

如果一个Agent写入了错误事实、代码或任务状态，仅重跑该Agent不够。恢复流程应先冻结相关任务，计算受影响对象，回滚或标记污染状态，撤销临时凭证，再从最后可信checkpoint重建上下文。对于不可逆外部动作，执行补偿流程并升级人工。

6. 可复用工程模式

模式	流程	适用	关键条件
Map-reduce research	独立搜索 -> 证据表 -> 汇总	研究/尽调	统一schema、去重和来源质量
Planner-worker-verifier	分解 -> 执行 -> 独立验收	长时任务	防止planner兼任自我裁判
Branch-and-merge coding	隔离分支 -> 契约测试 -> 合并	并行编码	接口先行和持续集成
Blackboard operations	事件/任务/状态共享，Agent订阅	异步多任务	版本、锁、权限和陈旧检测
Adversarial reviewer	生成者与攻击/审查者分离	安全和高风险产物	评审模型独立性和证据
Capability router	按工具、权限、成本和历史表现路由	大规模Agent服务	在线校准和fallback

设计检查表

- 每个Agent是否有真实不同的工具、信息、权限或目标？
- 是否定义了可机器校验的输入/输出schema和状态版本？
- 是否能在不广播完整轨迹的情况下传递最小充分上下文？
- 是否记录每个Agent的独特贡献、重复率和冲突率？
- 共享状态是否支持并发控制、来源和回滚？
- 失败后能否定位最早决定性步骤并清理污染？
- 单Agent强基线和等预算基线是否仍然落后？

7. 案例与代表论文精读卡

案例1: 多Agent深度研究

适合把独立来源、地区、时间段或假设分给子Agent。主管应先定义证据字段和排除标准，子Agent返回证据表与不确定性，汇总者做实体对齐和冲突检查。若直接让每个Agent写一篇长文再合并，会产生大量重复和无法追踪的二次表述。

案例2: 并行编码

并行的前提是模块边界、测试和分支隔离。C编译器团队实验说明，Agent可以在长时间内生产大量代码，但合并接口、全局架构和端到端测试决定最终质量。并行应围绕可独立验收的组件，而不是按文件随意切分。

案例3: 多任务办公室

CORPGEN表明多任务负载会使记忆互相干扰。可行架构是每个任务独立上下文与状态，主管只持有依赖图和里程碑；任务间通过明确产物链接，而不是共享对话历史。

P23 | 2025-10 | memory/multi-agent | 窗口内

LEGOMem: Modular Procedural Memory for Multi-Agent Workflows

主要贡献: OfficeBench results suggest orchestrator memory helps decomposition while task-agent memory improves execution accuracy.

阅读限制: Placement effects may shift with topology, model size and retrieval policy.

[原文](#)

P32 | 2025-11 | multi-agent/security | 窗口内

TAMAS: Adversarial Risks in Multi-Agent LLM Systems

主要贡献: Demonstrates that adversarial influence can propagate across role and message boundaries.

阅读限制: Threat success rates are topology- and defense-specific.

[原文](#)

P33 | 2025-11 | multi-agent/reliability | 窗口内

Detecting Silent Failures in Multi-Agent Trajectories

主要贡献: Treats silent failure as trajectory anomaly detection rather than final-answer error.

阅读限制: Detection does not automatically yield correct attribution or recovery.

[原文](#)

P58 | 2026-05 | multi-agent/architecture | 窗口内

Coordination as an Architectural Layer

主要贡献: Five controlled coordination configurations produce different calibration and cost signatures even with one fixed model.

阅读限制: $n=100$ results are directional and do not establish a universal best topology.

[原文](#)

P59 | 2026-05 | multi-agent/survey | 窗口内

Beyond Individual Intelligence: Collaboration, Attribution and Evolution

主要贡献: The LIFE progression links foundation, integration, failure attribution and self-evolution as one causal chain.

阅读限制: The framework is a research map, not a validated production lifecycle.

[原文](#)

P65 | 2026-07 | multi-agent/reliability | 窗口内

AgentLocate: Failure Localization in Multi-Agent Systems

主要贡献: Attributes both the responsible agent and earliest decisive step using multi-perspective verification.

阅读限制: LLM judges may share biases with the target system and need human calibration.

[原文](#)

本卷结论

多智能体系统的智能上限由信息架构和接口决定，可靠性下限由最差的共享状态与验证决定。先证明独立性和等预算增益，再增加Agent数量。